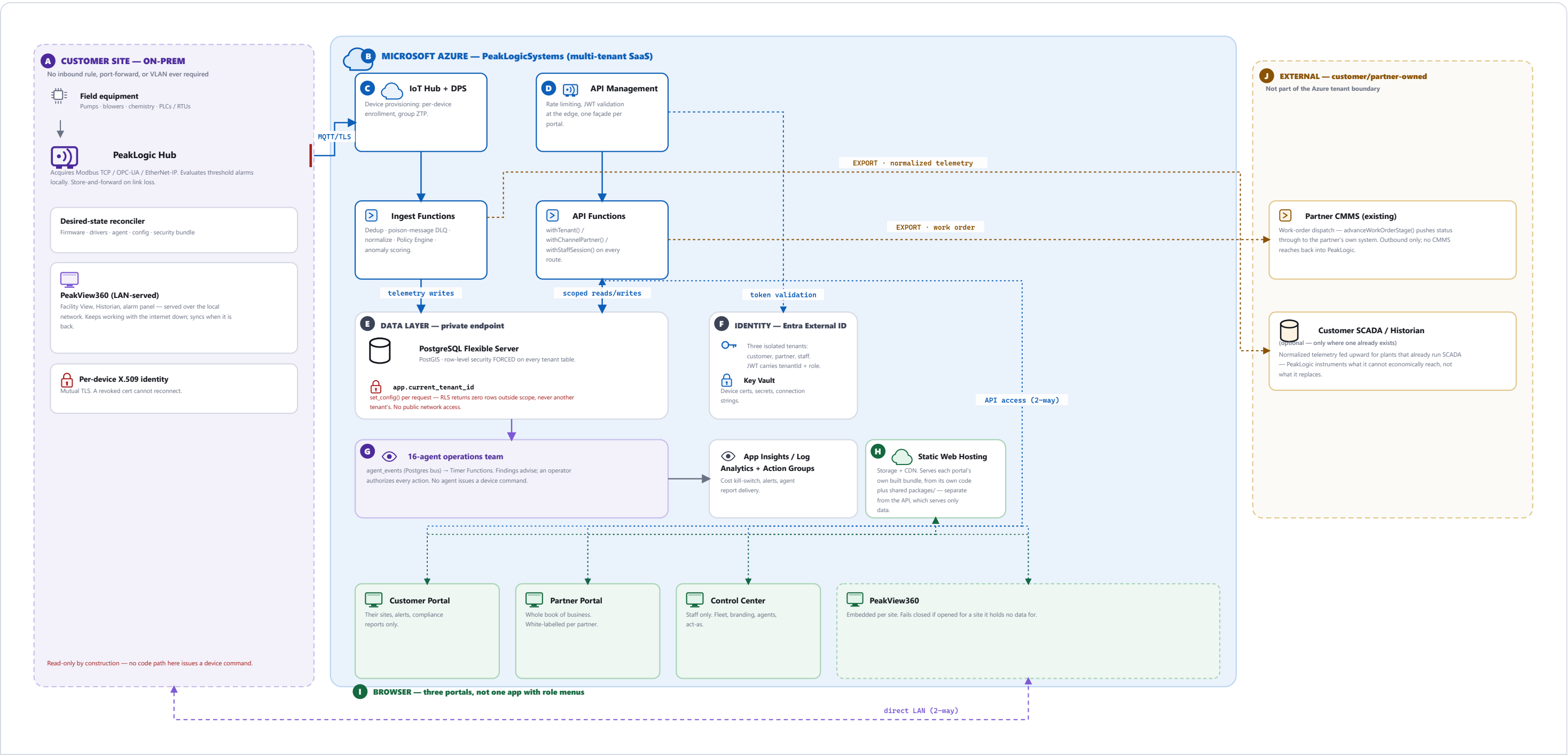


PeakLogic platform — system & data-flow diagram

Azure-native, multi-tenant. Every arrow below is a real data flow; every dashed boundary is an enforced isolation boundary, not an aspiration. Prepared for architectural and security review.



Reading the diagram

A fixed vocabulary of shapes, used consistently, plus a lettered badge on every boundary — cross-referenced below — so a review conversation can say "look at E" instead of re-pointing at the screen. One honest limitation: these are hand-drawn analogues of the well-known Azure shapes, not Microsoft's own licensed icon set, which this document's build process has no way to embed.

</

What a reviewer should take away

FLOW	PATH	SECURITY CONTROL
Telemetry ingest	Equipment → Hub → IoT Hub/DPS → Ingest Functions → PostgreSQL	MTLS per-device X.509 cert, outbound-only session, no inbound firewall rule ever required on the customer network. One direction only
Operator / customer request	Browser ⇌ APIM ⇌ API Functions ⇌ PostgreSQL	ENTRA JWT validated at APIM; every route wrapped in withTenant() / withChannelPartner() / withStaffSession(), which activates PostgreSQL row-level security. Genuine round trip — request and response
Static asset delivery	Browser → Storage + CDN (Static Web Hosting)	Separate from the API path above — this serves the SPA's code, not its data. No auth gate needed; nothing here is tenant-scoped
PeakView360 (embedded)	Portal iframe ⇌ Hub, direct over the site LAN, when opened on-site	Fail-closed scope guard — refuses to render if it holds no data for the requested site, rather than falling back to another site's facility. Two-way
Agent operations	PostgreSQL event bus → Timer Functions → Action Groups (email) / audit log	Findings carry tenant_id and are RLS-scoped. Agents propose only — every action needs operator authorization, logged to that operator, never to the agent
Work-order dispatch (export)	API Functions → PARTNER CMMS , outbound only	advanceWorkOrderStage() pushes ticket status through to the partner's own CMMS. Nothing in that system calls back into PeakLogic — the trust direction is one-way
Telemetry export (optional)	Ingest Functions → CUSTOMER SCADA/HISTORIAN , where one already exists	Normalized telemetry only, fed upward — PeakLogic instruments the long tail an existing SCADA cannot economically reach, it does not replace safety-rated control logic
Command / actuation	NOT BUILT	No code path publishes a device command today. The network model (outbound MQTT/TLS, per-device topic) already supports it; authorization, safety interlocks and audit trail are designed before any command path is opened

Tenant isolation

Every tenant-scoped query runs inside a transaction that sets `app.current_tenant_id` via `set_config()`, activating PostgreSQL RLS — **forced**, not merely enabled, and with no `BYPASSRLS` role. A site outside a viewer's scope is refused, never approximated.

Deployment reality

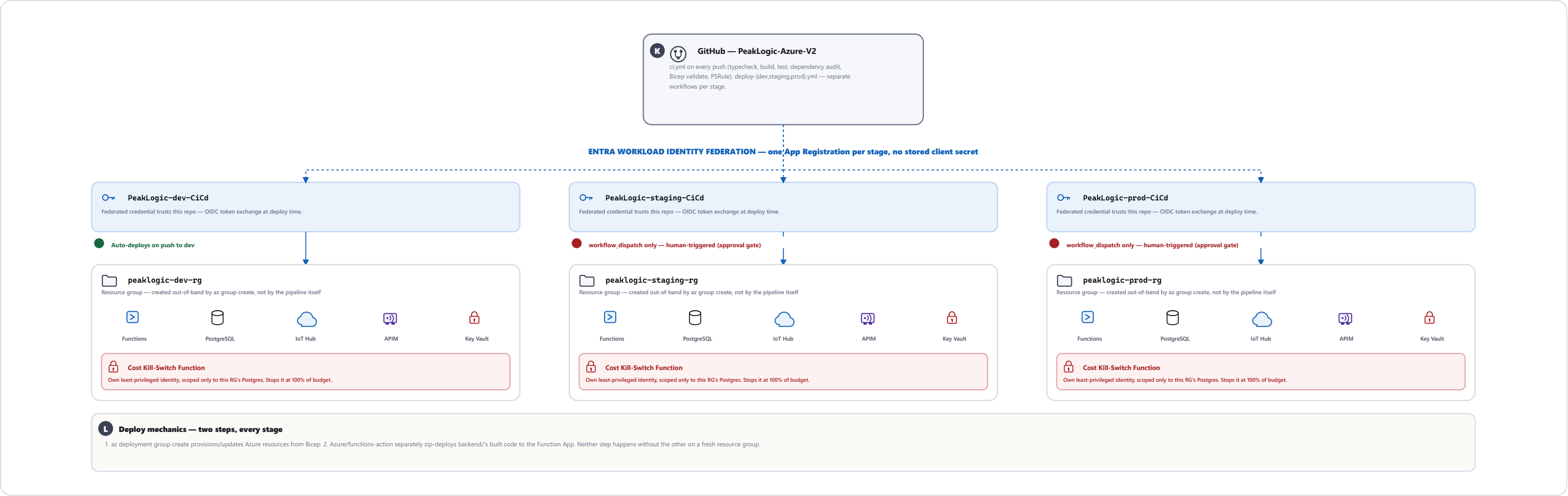
Defined in Bicep, validated by `az bicep build` and `PSRule` on every push. **NO ENVIRONMENT DEPLOYED YET** — this diagram describes the architecture as designed and CI-validated, not a running production system.

Cost containment

A dedicated Cost Kill-Switch Function (its own least-privileged identity, scoped only to the Postgres resource) stops the database at 100% of budget. Agents run on the existing consumption plan — additive spend is near \$0.

How the Azure tenant itself is organised

What this diagram shows — the deployment path: how code in source control becomes running Azure infrastructure, and what separates dev from production. Top to bottom: **(K)** the GitHub repository (CI on every push, one deploy workflow per stage) → **Entra Workload Identity Federation**, issuing each stage its own App Registration that trusts this repo over OIDC, so no client secret is ever stored → three **resource groups**, one per stage, each with a full copy of the platform's services and its own cost kill-switch → **(L)** the two-step mechanism every deploy performs. The coloured dot above each resource group is its trigger: dev deploys automatically; staging and prod need a human to start the run. Pages 1–3 cover how data moves through the running system; this page covers how that system gets deployed.



NOT DEPLOYED No environment has been created yet — `peaklogic-{dev,staging,prod}-rg`, the Entra App Registrations, and the federated credentials are all still to-do, not running infrastructure. This page describes the designed target, same disclosure as pages 1–3.